

Cấu trúc dữ liệu **Heap**

Học phần Cấu trúc dữ liệu và Giải thuật 2

TS Phạm Hữu Tùng

Giới thiệu về Heap

● Cấu trúc dữ liệu cơ bản

➤ Heap là gì?

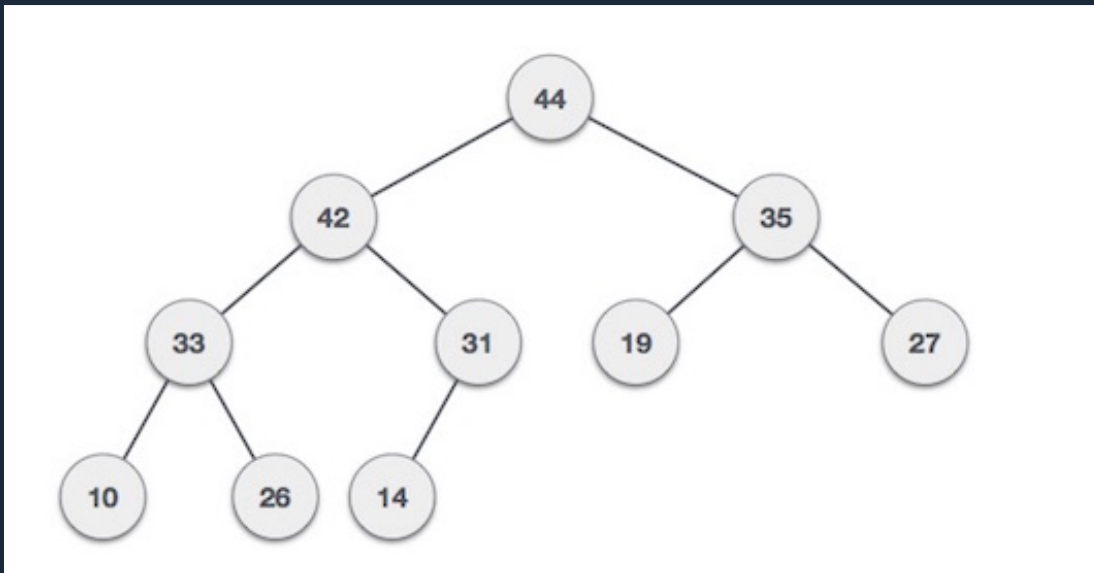
Heap là một cấu trúc dữ liệu đặc biệt dựa trên cây.

Luôn thỏa mãn các tính chất của Heap (Heap Property).

Thường dùng để triển khai hàng đợi ưu tiên (Priority Queues).

Cây nhị phân

Truy cập hiệu quả



Max Heap: Phần tử lớn nhất luôn nằm ở gốc.

Đặc điểm chính của Heap

- ✓ Là Cây nhị phân gần đầy (Complete Binary Tree).
- ✓ Mọi cấp độ đều được lấp đầy, trừ cấp cuối cùng.
- ✓ Các nút ở cấp cuối cùng được lấp đầy từ trái sang phải.

↑ 1. Max Heap Property

Giá trị nút cha lớn hơn hoặc bằng giá trị các nút con.

Phần tử lớn nhất ở gốc

↓ 2. Min Heap Property

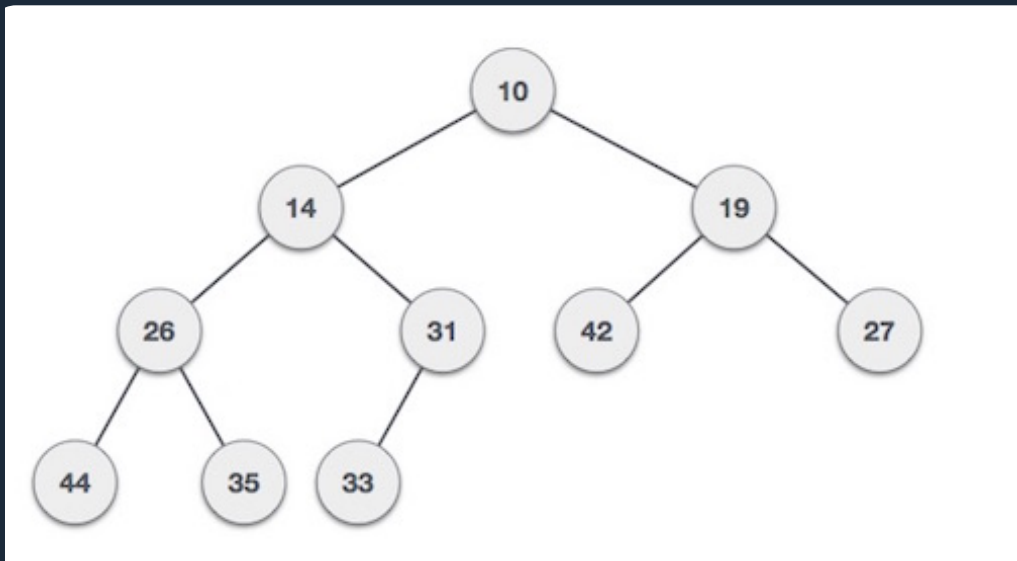
Giá trị nút cha nhỏ hơn hoặc bằng giá trị các nút con.

Phần tử nhỏ nhất ở gốc

Cài đặt Heap bằng C++: Biểu diễn

🏠 Biểu diễn Heap bằng mảng

Vì Heap là một cây nhị phân gần đầy, ta có thể biểu diễn nó một cách hiệu quả bằng mảng mà không cần dùng đến con trỏ. Cách lưu trữ này tiết kiệm không gian và cho phép truy cập nhanh các phần tử.



⇔ Mối quan hệ chỉ số Cha-Con

↑ Cha của phần tử i : $(i - 1) / 2$

↓ Con trái của phần tử i : $2 * i + 1$

↓ Con phải của phần tử i : $2 * i + 2$

Lưu ý: Công thức áp dụng cho mảng có chỉ số bắt đầu từ 0.

✅ Ưu điểm

Tiết kiệm bộ nhớ (không cần con trỏ).

Truy cập nhanh nhờ tính toán chỉ số.

Cài đặt và duyệt cache hiệu quả hơn.

⊗ Nhược điểm

Kích thước mảng cố định, việc thay đổi kích thước khá tốn kém.

Các phép toán cơ bản trên **Heap**

Heapify (Vun đống)

- Điều chỉnh một nút để duy trì tính chất Heap.
- So sánh nút cha với các nút con, hoán đổi nếu cần.
- Độ quy vun đống xuống nút bị ảnh hưởng.

Core Operation

Rebalance

Insert (Thêm phần tử)

- Thêm phần tử mới vào cuối cấu trúc.
- Thực hiện "vun đống lên" (heapify-up) để điều chỉnh.
- Đảm bảo vị trí đúng trong Heap, bảo toàn tính chất.

Add Node

Upward Adjustment

Delete (Xóa phần tử)

- Thường xóa phần tử gốc (lớn nhất/nhỏ nhất).
- Thay thế gốc bằng phần tử cuối cùng của Heap.
- Thực hiện "vun đống xuống" (heapify-down) để duy trì.

Remove Root

Downward Adjustment

Build Heap (Xây dựng)

- Chuyển đổi một mảng tùy ý thành cấu trúc Heap.
- Áp dụng heapify từ nút không-phải-lá cuối cùng ngược lên gốc.
- Hiệu quả hơn việc insert từng phần tử, độ phức tạp $O(N)$.

Initialization

Efficient $O(N)$

Cài đặt Heap bằng C++: Hàm Heapify

Mã nguồn hàm `heapify` (Max Heap)

```
void heapify(int arr[], int n, int i) {
    int largest = i;          // Khởi tạo largest là root
    int left = 2 * i + 1;     // left child = 2*i + 1
    int right = 2 * i + 2;    // right child = 2*i + 2

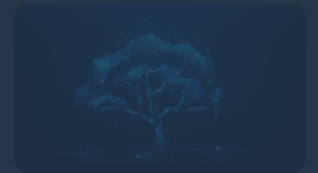
    // Nếu con trái lớn hơn root
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // Nếu con phải lớn hơn largest hiện tại
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // Nếu largest không phải là root
    if (largest != i) {
        swap(arr[i], arr[largest]);
        // đệ quy heapify subtree bị ảnh hưởng
        heapify(arr, n, largest);
    }
}
```

Giải thích chi tiết từng bước

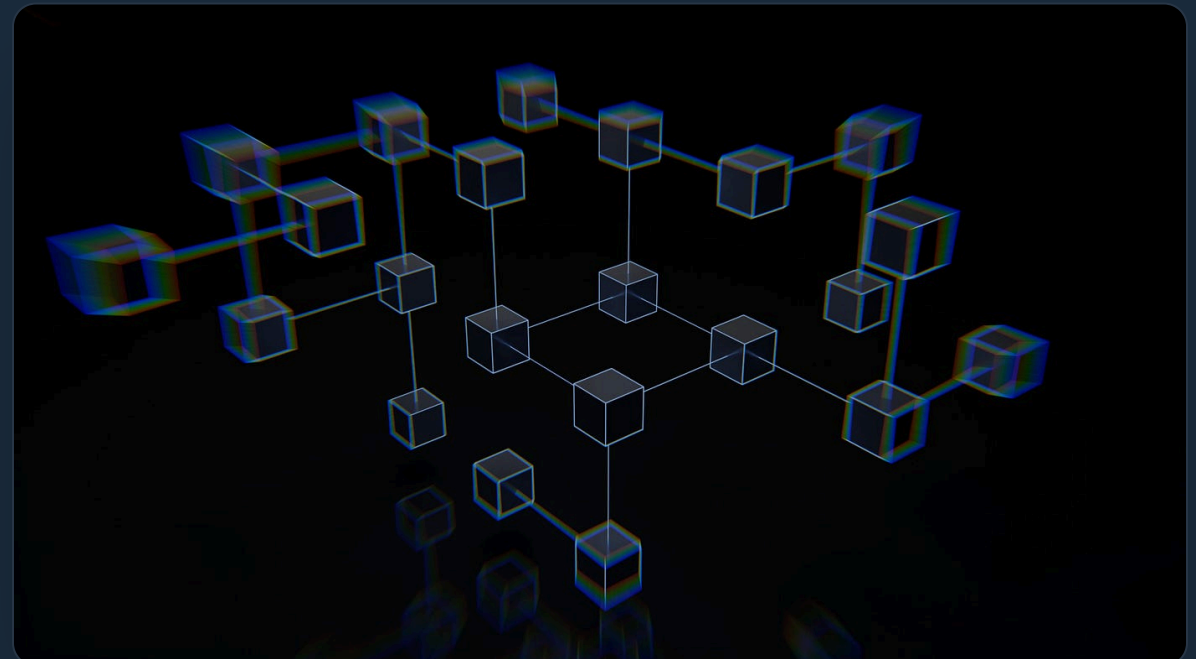
- ▶ **Khởi tạo:** Đặt `largest` là nút gốc của cây con cần xét (chỉ số `i`).
- **Xác định con:** Tính chỉ số của con trái ($2*i + 1$) và con phải ($2*i + 2$).
- ↔ **So sánh:** Tìm ra phần tử lớn nhất trong bộ ba: nút gốc, con trái và con phải. Cập nhật `largest`.
- ↔ **Hoán đổi & Đệ quy:** Nếu nút gốc không phải lớn nhất, hoán đổi nó với phần tử lớn nhất và gọi đệ quy `heapify` trên cây con bị ảnh hưởng.
- ✓ **Kết quả:** Đảm bảo cây con có gốc `i` tuân thủ tính chất Max Heap.



Ví dụ minh họa quá trình `heapify`

- Trạng thái ban đầu có thể phá vỡ tính chất Heap.
- Hàm `heapify` được gọi để "sửa chữa" cấu trúc từ trên xuống.
- Quá trình đảm bảo nút cha luôn lớn hơn các nút con.

🔗 Nguyên lý thích nghi ⚙️ Chu trình điều chỉnh



Cài đặt Heap bằng C++: Hàm Insert

Quá trình 'Sift-Up' hoặc 'Bubble-Up'

Bước 1: Thêm vào cuối

Chèn phần tử mới vào vị trí trống đầu tiên ở cuối Heap để duy trì thuộc tính cây nhị phân gần đầy.



Bước 2: So sánh và đổi chỗ

So sánh phần tử mới với nút cha của nó. Nếu vi phạm tính chất Heap (ví dụ: con > cha trong Max-Heap), hoán đổi chúng.



Bước 3: Lặp lại quá trình

Tiếp tục quá trình "sift-up" (trôi lên) cho đến khi phần tử mới ở đúng vị trí (không vi phạm tính chất Heap) hoặc nó đã trở thành nút gốc.

Thêm phần tử

Max-Heap

Sift-Up

🕒 Độ phức tạp thời gian



$O(\log N)$ - vì quá trình sift-up chỉ di chuyển trên một nhánh của cây, có chiều cao là $\log N$.

Mã nguồn C++ cho hàm `insert`

```
void insert(int value) {
    // Bước 1: Thêm phần tử vào cuối vector
    heap.push_back(value);
    int currentIndex = heap.size() - 1;

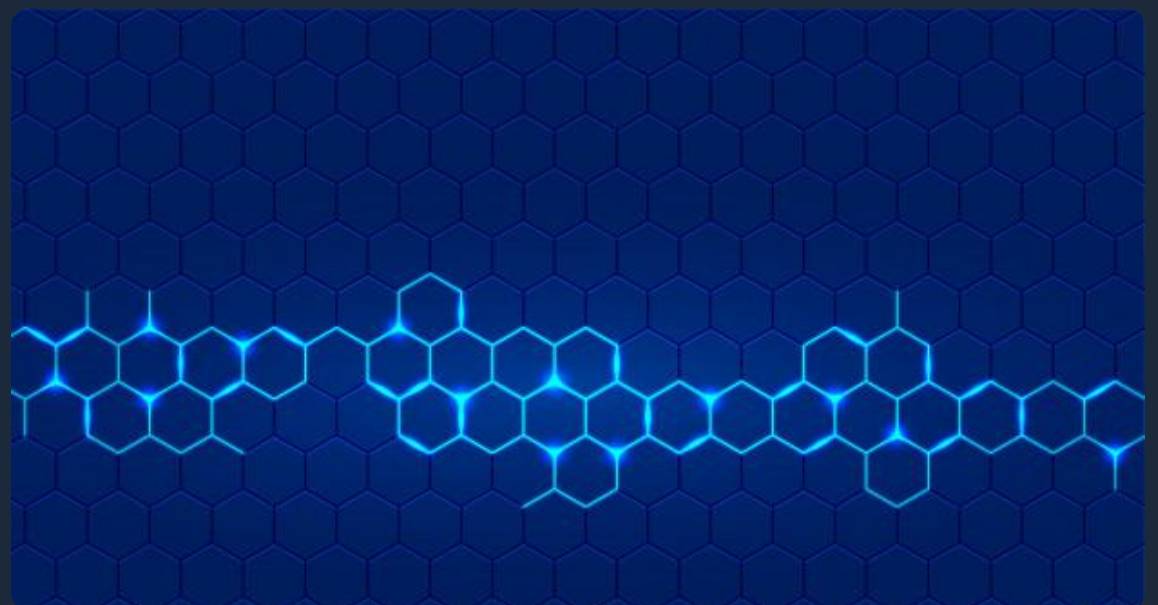
    // Bước 2 & 3: Thực hiện "sift-up"
    while (currentIndex > 0 && heap[currentIndex] >
heap[parent(currentIndex)]) {
        // Đổi chỗ với cha nếu lớn hơn (đối với Max-Heap)
        std::swap(heap[currentIndex],
heap[parent(currentIndex)]);

        // Di chuyển lên vị trí của cha để tiếp tục so
sánh
        currentIndex = parent(currentIndex);
    }
}
```

C++

Vector

Array-based Heap



Cài đặt Heap bằng C++: Hàm Delete

07

Quá trình Xóa & 'Sift-Down'

-  Xóa phần tử gốc (lớn nhất trong Max-Heap hoặc nhỏ nhất trong Min-Heap).
-  Đưa phần tử cuối cùng của Heap lên thay thế vị trí của gốc vừa xóa.
-  Sử dụng 'sift-down' (heapify) để di chuyển phần tử mới xuống đúng vị trí, bảo toàn tính chất Heap.



Độ phức tạp thời gian

 $O(\log N)$

Số bước 'sift-down' tối đa bằng chiều cao của cây.

Mã nguồn C++: `deleteMax`

```
int deleteMax() {  
    // Nếu heap rỗng hoặc chỉ có 1 phần tử  
    if (heap_size <= 0) return INT_MIN;  
    if (heap_size == 1) {  
        heap_size--;  
        return harr[0];  
    }  
  
    // 1. Lưu giá trị gốc (lớn nhất)  
    int root = harr[0];  
  
    // 2. Thay gốc bằng phần tử cuối  
    harr[0] = harr[heap_size - 1];  
    heap_size--;  
  
    // 3. Gọi heapify (sift-down) để tái cân bằng  
    heapify(0);  
  
    return root;  
}
```

Trực quan hóa 'Sift-Down'

"Sift-down" đảm bảo nút cha luôn lớn hơn các nút con của nó.

 Đảm bảo tính toàn vẹn

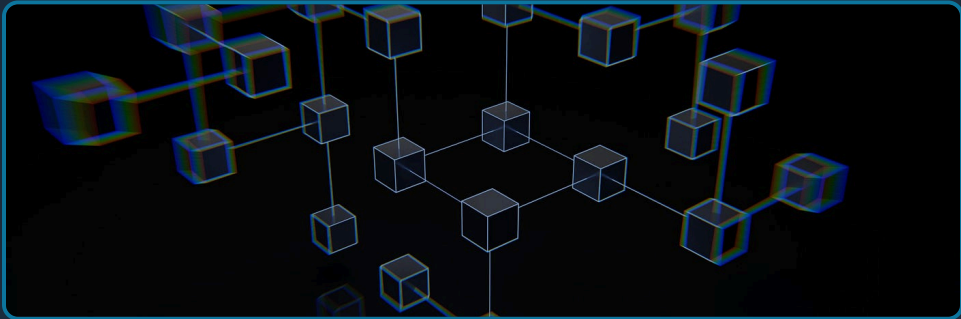
Cài đặt Heap bằng C++: Hàm Build Heap

Hàm `buildHeap` tối ưu hóa quá trình xây dựng Heap từ một mảng không có thứ tự, đảm bảo tính chất Heap một cách hiệu quả.

Mã nguồn `buildHeap`

```
// Xây dựng Max Heap từ một mảng cho trước
void buildHeap(std::vector<int>& arr) {
    int n = arr.size();
    // Bắt đầu từ nút cha không phải lá cuối cùng
    // và vun đống xuống cho tất cả các nút cha
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

// (Hàm heapify(arr, n, i) được giả định đã cài
// Nó sẽ vun đống một cây con có gốc tại chỉ số i)
```



Minh họa cây nhị phân gần đầy được vun đống

Thuật toán xây dựng Heap hiệu quả ($O(N)$)

↓ Tiếp cận từ dưới lên (Bottom-up):

Bắt đầu quá trình `heapify` từ nút cha không phải lá cuối cùng và đi ngược lên gốc.

⌚ Lợi thế:

Các nút ở mức thấp (có số lượng nhiều nhất) chỉ cần ít thao tác `sift-down` để ổn định, giúp giảm tổng chi phí.

Σ Phân tích độ phức tạp:

Tổng công việc có thể chứng minh là tuyến tính, cho độ phức tạp thời gian là $O(N)$.

Tối ưu hóa

Phân tích độ phức tạp

So sánh các phương pháp xây dựng Heap

✅ `buildHeap` ($O(N)$)

Xây dựng tại chỗ từ mảng. Gọi `heapify` từ dưới lên. Rất hiệu quả cho dữ liệu tĩnh ban đầu.

Hiệu suất

Tiết kiệm thời gian

⚡ Chèn từng bước ($O(N \log N)$)

Thêm lần lượt N phần tử vào heap. Mỗi lần chèn tốn $O(\log N)$. Kém hiệu quả hơn cho việc khởi tạo.

Ứng dụng của Heap

Cấu trúc dữ liệu mạnh mẽ cho các bài toán tối ưu

☰ Thuật toán Heap Sort

Sắp xếp dữ liệu hiệu quả dựa trên cấu trúc Heap. Biến đổi mảng thành Heap rồi trích xuất phần tử.

Sắp xếp

Tại chỗ

$O(n \log n)$

📍 Thuật toán đồ thị

Dùng trong các phiên bản của Dijkstra (đường đi ngắn nhất) và Prim (cây bao trùm nhỏ nhất).

Đồ thị

Tối ưu

Dijkstra & Prim

! Hàng đợi ưu tiên

Cấu trúc lấy ra phần tử có độ ưu tiên cao nhất/thấp nhất. Heap là cách triển khai tối ưu nhất.

Hàng đợi

Ưu tiên

Thêm/Xóa $O(\log n)$

🔍 Tìm k phần tử lớn nhất/nhỏ nhất

Tìm nhanh k phần tử trong một tập hợp mà không cần sắp xếp toàn bộ dữ liệu.

Lọc dữ liệu

Truy vấn

Chọn lọc

"Heap là một cấu trúc dữ liệu dạng cây đặc biệt, thường được dùng để triển khai hàng đợi ưu tiên, nơi phần tử lớn nhất (hoặc nhỏ nhất) luôn nằm ở gốc."

- GeeksforGeeks

Complete Binary Tree

Max/Min Heap Property

Tham khảo: GeeksforGeeks, Programiz

Tổng kết về Heap và Câu hỏi



Kiến thức chính về Heap

Là cấu trúc dữ liệu cây nhị phân gần đầy.

Tuân theo tính chất Max-Heap hoặc Min-Heap.

Phần tử gốc luôn là lớn nhất (Max-Heap) hoặc nhỏ nhất (Min-Heap).

Thường được biểu diễn bằng mảng để tiết kiệm không gian.

• Cây nhị phân

• Priority Queue

• Heap Sort



Một số hạn chế

Không tối ưu cho việc tìm kiếm một phần tử bất kỳ ($O(N)$).

Chỉ cho phép truy cập nhanh vào phần tử gốc.

Duyệt cây theo thứ tự không trực quan như cây tìm kiếm.

• Giới hạn truy cập

• Không tự cân bằng



Ưu điểm nổi bật

Nền tảng hiệu quả cho Hàng đợi ưu tiên (Priority Queue).

Sắp xếp tại chỗ (Heap Sort) với độ phức tạp $O(N \log N)$.

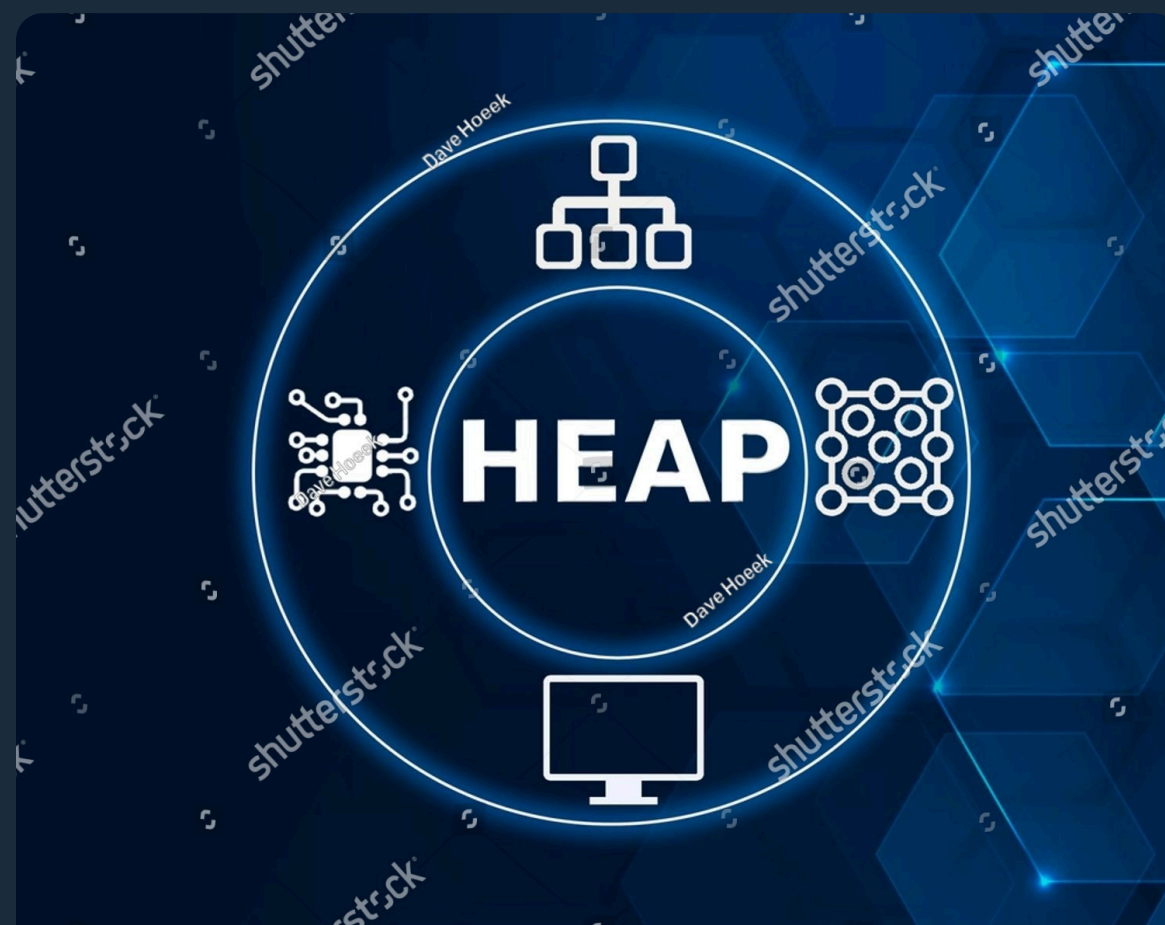
Xây dựng heap từ mảng chỉ mất $O(N)$.

Tìm phần tử lớn/nhỏ nhất trong $O(1)$.

• Hiệu suất cao

• Ổn định

• Đa dụng



shutterstock

IMAGE ID: 2644869761
www.shutterstock.com



Hãy cùng thảo luận về Cấu trúc dữ liệu Heap!

✉ Liên hệ: tungph@huce.edu.vn

Source: GeeksforGeeks, Programiz, Studocu, VisuAlgo